Name: Priyanka M Reddy

SRN: PES2UG24CS378

Section: G

Pylint :

```
PS C:\Users\priya\Downloads\L5_10_parking_permits-main\L5_10_parking_permits-main> python -m pylint .\parking_system.py

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Bandit:

```
PS C:\Users\priya\Downloads\L5_10_parking_permits-main\L5_10_parking_permits-main> python -m bandit -r .\parking_system.p
y
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.15
Run started:2026-10-16 04:23:48.778007+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 98
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
```

Flake:

```
PS C:\Users\priya\Downloads\L5_10_parking_permits-main\L5_10_parking_permits-main> python -m flake8 .\parking_system.py
PS C:\Users\priya\Downloads\L5_10_parking_permits-main\L5_10_parking_permits-main>
```

Reflection

1. Which issues were easiest and hardest to fix, and why?

The easiest issue which were straightforward to change were to fix the formatting problem, such as missing blank lines, long lines and inconsistent function naming.

The hardest issues were handling exceptions correctly, removing the mutable default argument, improving file handling, and validating user inputs. These required understanding how the code behaved and how the changes affected its reliability.

2. Did the static analysis tools report any false positives? If so, describe one.

The tools identified potential problems that needed to be evaluated in the context of the program. For example, Bandit flagged the use of Python's random module for generating permit numbers. Since this module is not suitable for security-sensitive random values, it was replaced with the secrets module. This was a valid warning rather than a clear false positive.

After fixing the issues, the final Bandit scan reported no issues.

3. How can static analysis tools be integrated into an actual development workflow?

Static analysis tools can be run regularly during development and before committing code to GitHub. Pylint can check code quality, Flake8 can check style violations, and Bandit can identify potential security problems.

These tools can also be integrated into a CI/CD pipeline, such as GitHub Actions, so that code is checked automatically whenever changes are pushed or a pull request is created.

4. What tangible improvements did you observe in code quality, readability, and robustness?

The code became more readable through improved naming, formatting, and documentation. Robustness improved by replacing the mutable default argument, validating parking zones and vehicle plates, handling exceptions more specifically, and using context managers for file operations.

The expiry date format was made consistent, and secrets replaced random for generating permit numbers.

Pylint's score improved from 6.32/10 to 10.00/10. The final Bandit scan reported zero issues, and Flake8 reported no violations.